Computer Networks – Project Report
Family Locator and Safety Network

Gavril Olivia-Alexandra

"Alexandru Ioan Cuza" University of Iaşi
Faculty of Computer Science

Abstract. This paper presents the *Family Locator and Safety Network*, a client–server system developed to enhance family coordination and safety through real-time location sharing and emergency alerts. The core architecture is built on **TCP sockets**, ensuring reliable communication, while server-side concurrency is efficiently managed using the **select()** mechanism. The application integrates a **relational SQL database** for robust data persistence and features an intuitive graphical user interface implemented with *raylib* and *Dear ImGui*.
Keywords: client–server, TCP sockets, select(), SQL database, graphical interface

1 Introduction

The *Family Locator and Safety Network* project is a client–server application created to help family members stay connected through location sharing and emergency alerts. The system uses a centralized architecture, where a main server manages all connected users, handles their requests, and coordinates how data is shared within each family group.

The application allows users to create accounts, log in, and organize themselves into family groups. Once connected, they can choose to share their location or request the position of other family members in real-time. Additionally, the system includes an SOS button, designed to send an instant notification to everyone in the family during an emergency.

The main objective of this project is to build a reliable system for communication over TCP, focusing on how the server handles multiple clients at the same time and processes commands corectly. By combining socket programming with a database and a graphical interface, the application becomes a practical tool for family coordination, ensuring that data is stored safely and the user experience is straightforward and effective.

2 Applied Technologies

The project is built as a simple client–server prototype that communicates through TCP sockets. TCP was chosen because it provides reliable and ordered delivery of data, which simplifies the implementation of the command-based communication protocol. This guarantees that the data transferred remains intact and arrives in the same order in which it was sent. Since the server must correctly interpret every request sent by the clients, TCP ensures a stable foundation for this type of structured interaction.

Concurrency on the server side is handled using the select() mechanism. This allows the server to manage multiple clients within a single process by monitoring all active sockets simultaneously.

To keep information safe even after the server is turned off, the system uses a SQL database to store user accounts, family groups, and location history. For the user-facing side, the application includes a graphical interface built with *raylib* and *Dear ImGui*. This makes the interaction much easier, replacing terminal commands with simple windows and buttons.

3 Application Architecture

The system uses a centralized architecture where the server acts as the core component. Every client connects to the server via TCP, and from that point, the server takes over: it listens for requests, checks if the data is valid, and sends back the appropriate responses.

Inside the server, each connected user is managed through a data structure that tracks things like whether they are logged in, which family they belong to, and their current location settings. While the server keeps track of these active details in its memory, all important information is saved permanently in a database so it is not lost.

The client's job is kept minimal: it only handles the user interface and sends the user's commands to the server. By keeping all the validation and coordination logic on the server side, the system ensures that everyone stays synchronized and the rules are applied consistently.

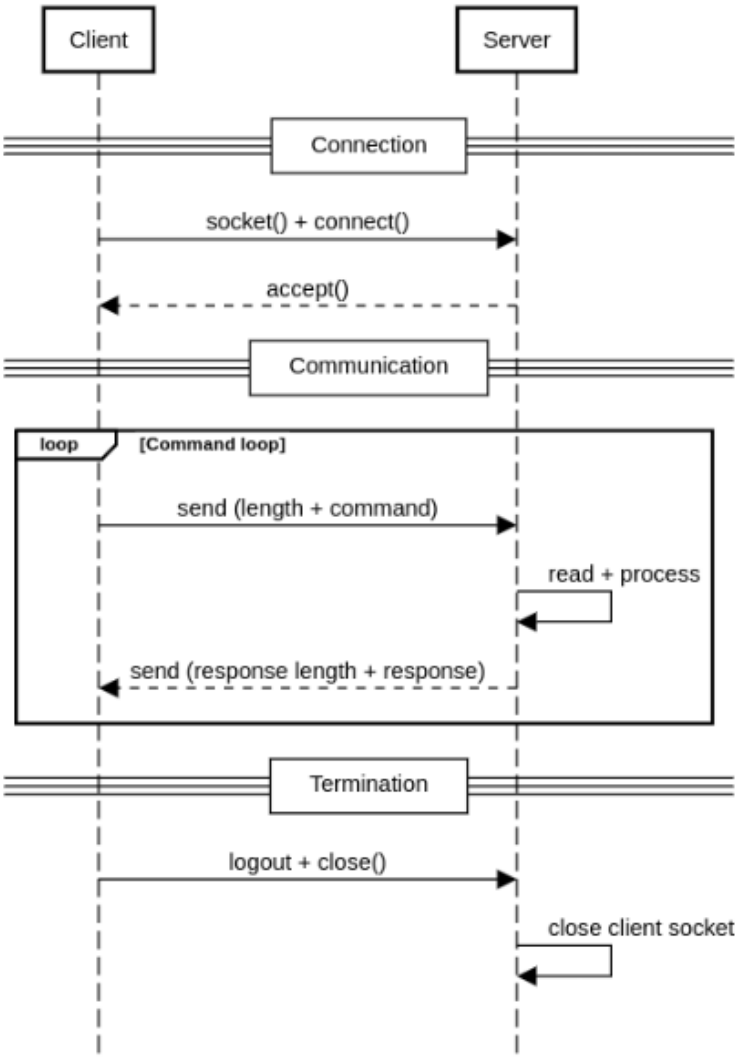


Fig. 1: Client–server communication using TCP

4 Application Flow

Figure 2 illustrates the general navigation flow of the application. The user begins on the login page and, after successful authentication, is redirected to the main page. From there, the user can choose from multiple options.

Switching between families triggers an update of the data displayed on the main page, ensuring that the information presented is always consistent with the currently selected family.

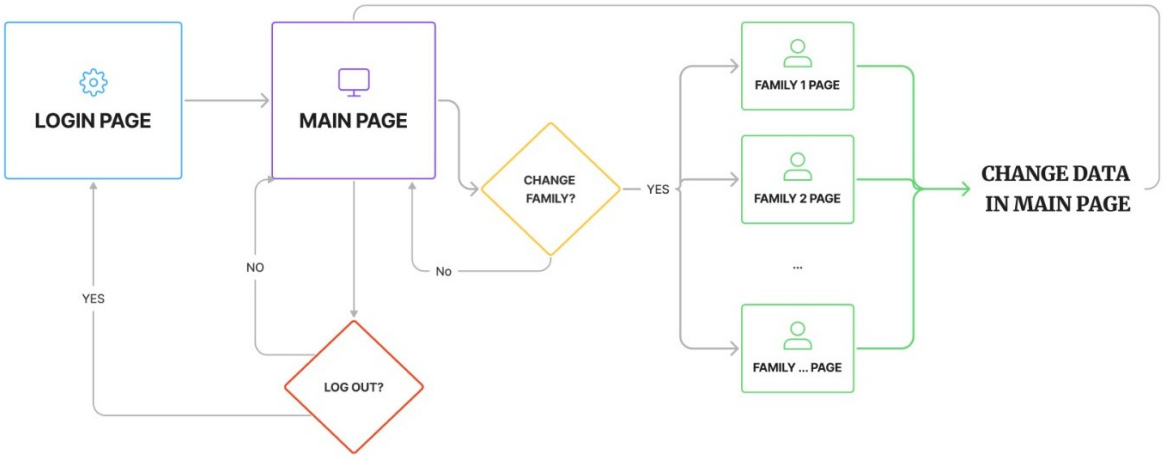
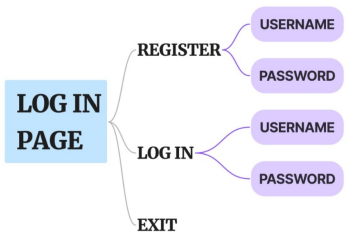


Fig. 2: Overall application flow and navigation

5 User Interface

5.1 Login and Registration

The login page provides options for user registration, authentication, and application exit. During registration, the user must provide a valid username and password. During login, the credentials are verified against the stored data.



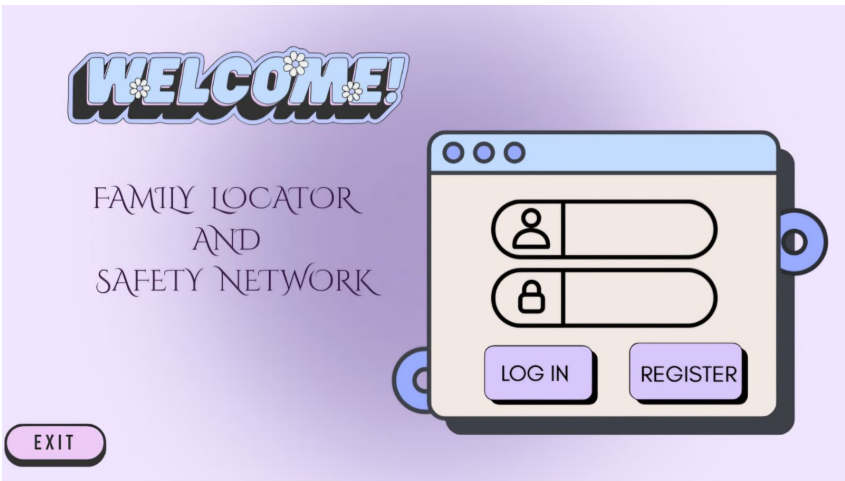


Fig. 3: Login Page Graphical Interface

5.2 Main Page

The main page represents the core interaction area of the application. From here, users can manage everything related to their family groups, location sharing and general settings.

The interface makes it easy to create a new family or join an existing one, add new members, or leave a group if needed. Users also have direct control over their privacy, with the ability to turn location sharing on or off at any time, while also being able to see where their family members are on the map.

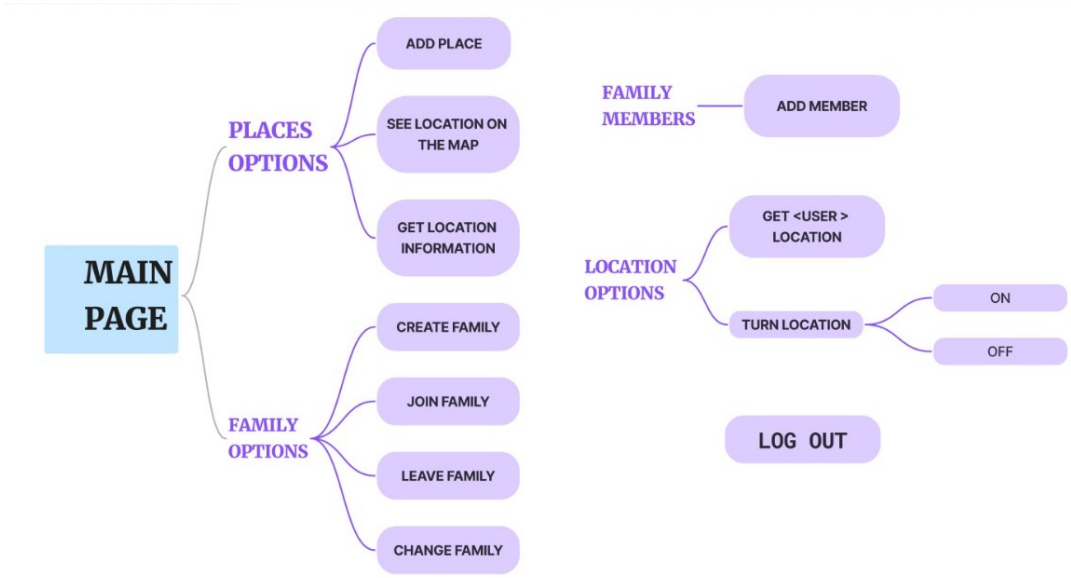


Fig. 4: Main page functionalities

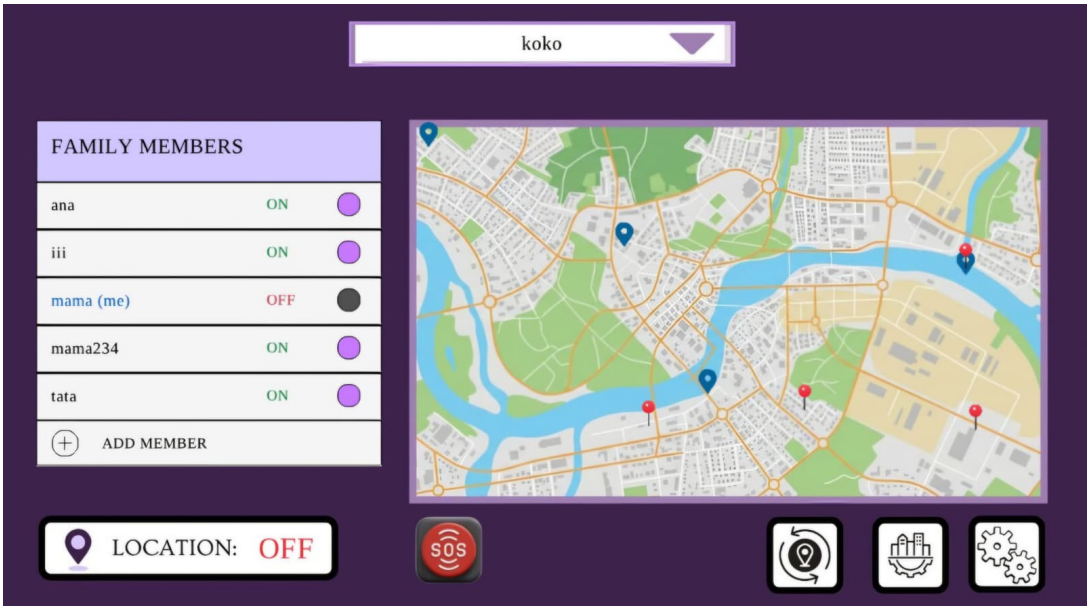


Fig. 5: Main Page Graphical Interface

6 Use Cases

Scenario 1: User Interaction

A new user registers, logs in, and creates a family group where they can add members and define specific locations. When the user reaches one of these areas, the system automatically notifies the family. Users can request location updates at any time or disable sharing completely to ensure privacy; once turned off, their position is no longer visible to others.

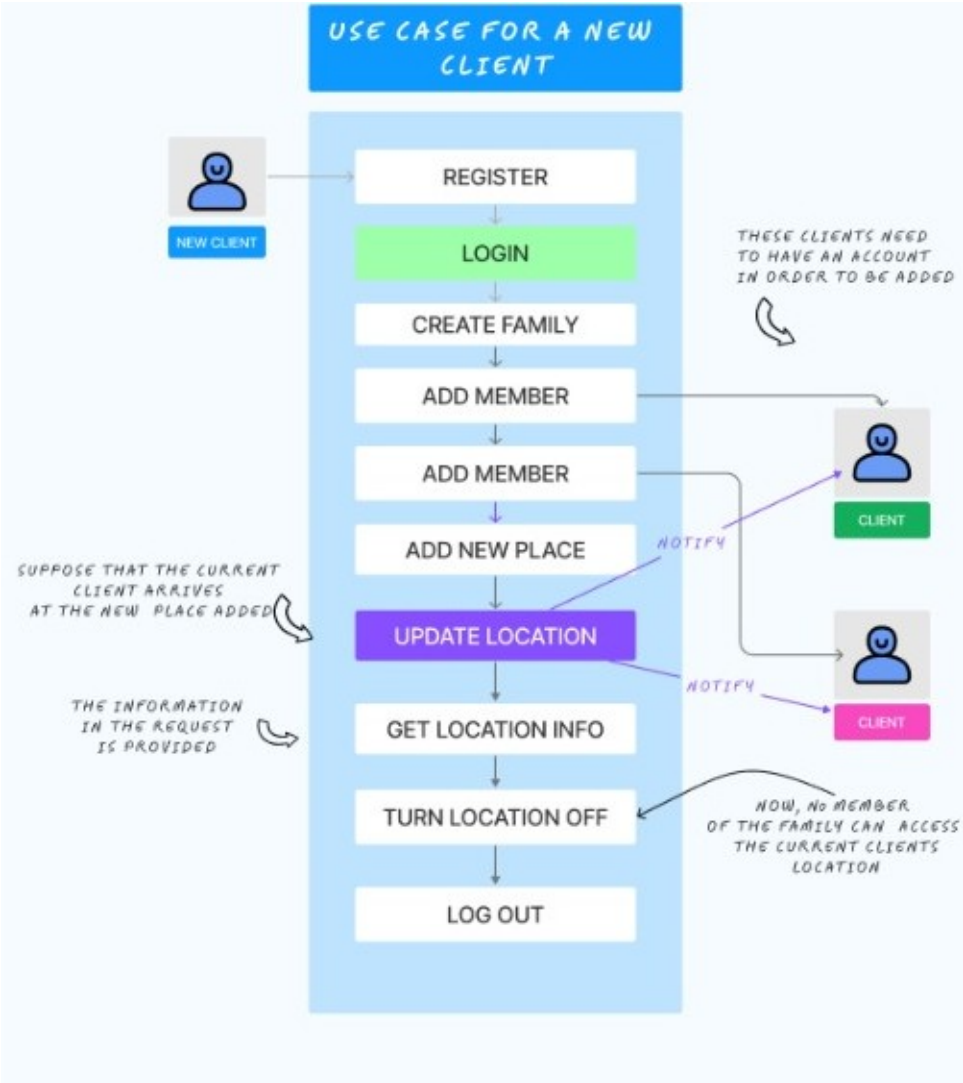


Fig. 6: New user scenario

Scenario 2 : SOS Alert

When a user triggers an SOS alert, the server notifies all family members instantly. The notification contains the sender's name, allowing the rest of the group to know immediately who needs assistance.

Scenario 3 : Basic Functionalities

Users can join families, define locations, and request member positions. Leaving the group instantly revokes access to all shared data.

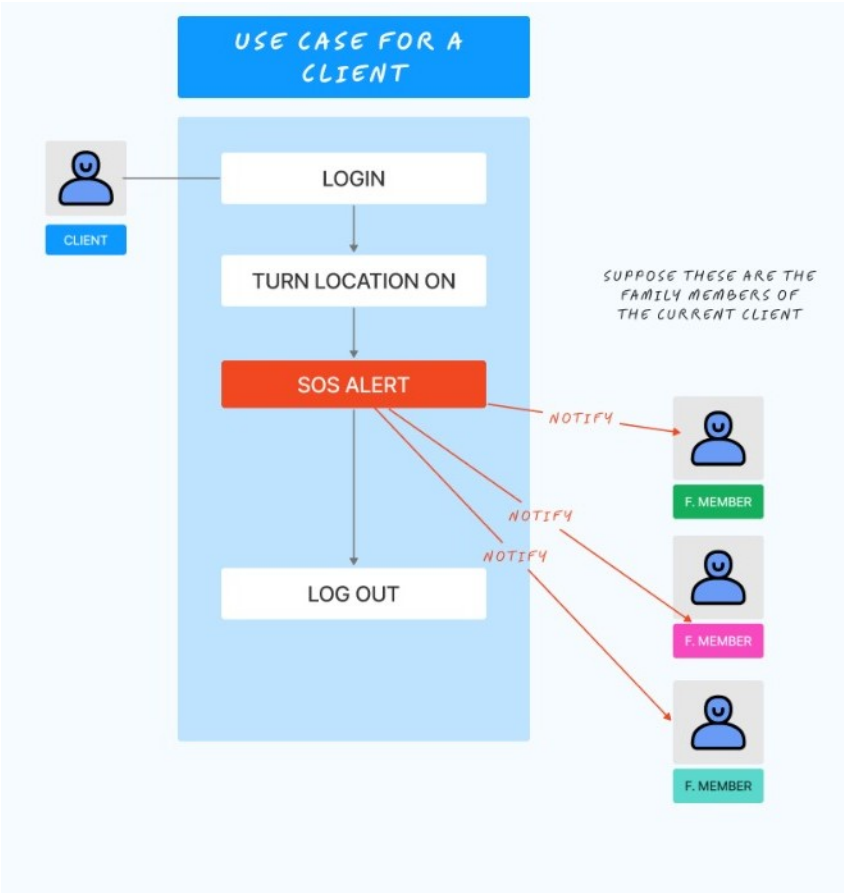


Fig. 7: SOS alert use case

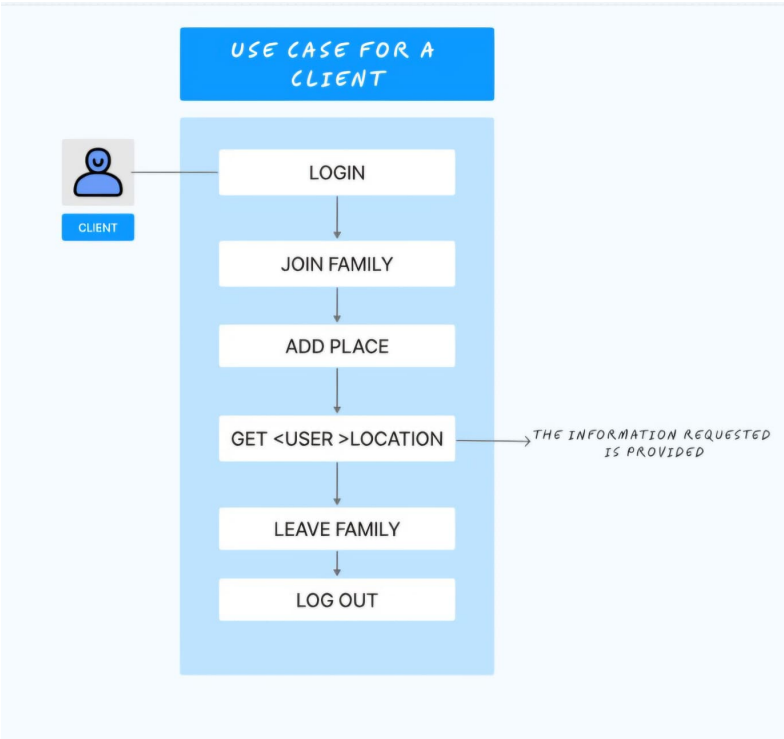
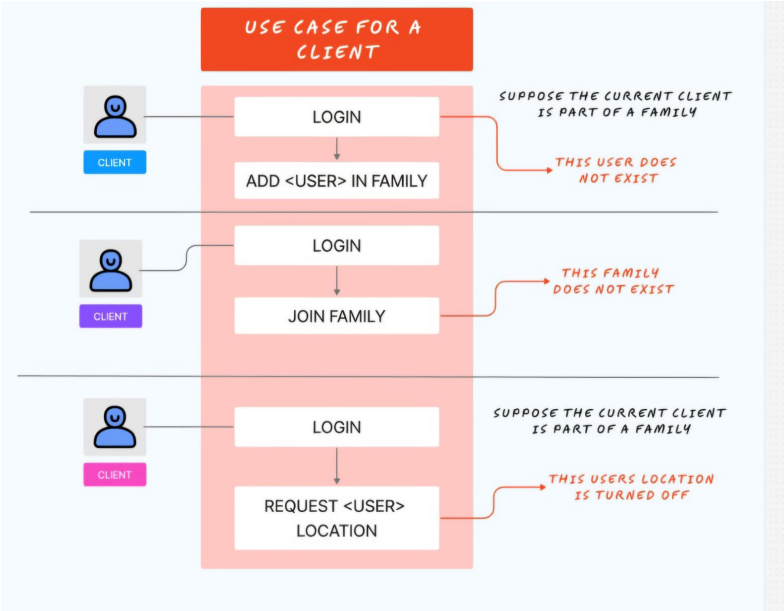


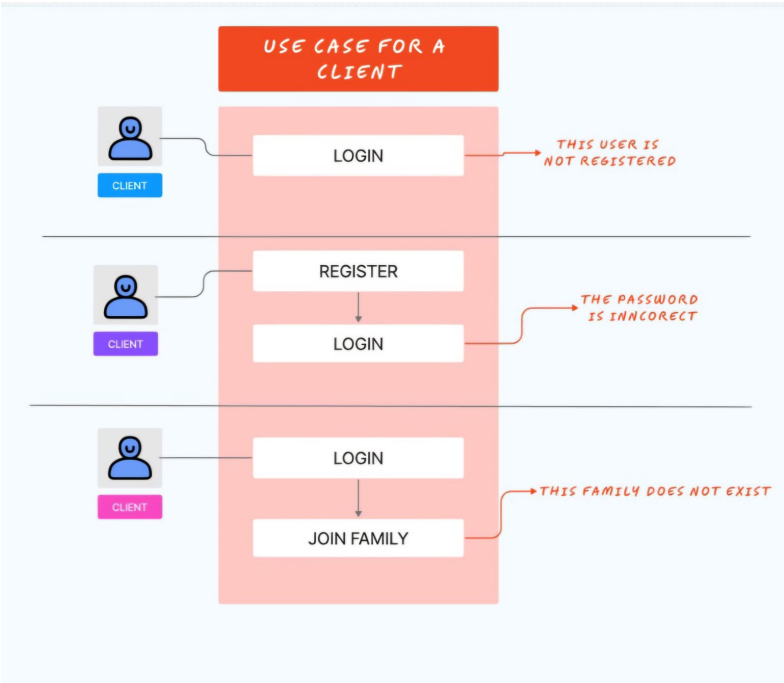
Fig. 8: Basic Scenario

Failure Scenarios

The system handles errors such as invalid logins or incorrect credentials by sending clear feedback to the user. This ensures that the application remains stable and the user is informed about what went wrong.



(a)



(b)

Fig. 9: Failure scenarios handled by the system

7 Conclusions and Future Work

The *Family Locator and Safety Network* project demonstrates a functional client–server system built on stable TCP communication and efficient concurrency management. By combining a SQL database with a graphical interface, the application moves beyond a simple prototype, becoming a practical tool for family safety.

Further improvements could be made with focus on adding real-time map integration, mobile platform support, and end-to-end encryption for user data. These features would make the network more practical for daily use and improve overall security.

References and Resources

1. Alboaie Lenuta, Panu Andrei, "TCP/IP server and client implementations, with I/O multiplexing using select().", Computer Networks -Faculty of Computer Science, Available at: https://edu.info.uaic.ro/computer-networks/files/NetEx/S9/servTcpCSel.c
2. Alboaie Lenuta, "Programarea in retea (III)", Computer Networks -Faculty of Computer Science.
3. GeeksforGeeks, "Centralized Architecture in Distributed System", 23 July 2025. Available at:https://www.geeksforgeeks.org/computer-networks/centralized-architecture-in-distributed-system/
4. TutorialsPoint, "Implementing SQLite with C/C++", available at: https://www.tutorialspoint.com/sqlite/sqlite_c_cpp.htm
5. GeeksforGeeks, "Practical SQL usage in C/C++", available at: https://www.geeksforgeeks.org/cpp/sql-using-c-c-and-sqlite/
6. W3Resource, "Handling dates and time in SQLite", available at: https://www.w3resource.com/sqlite/sqlite-strftime.php
7. SQLite Documentation, "C/C++ Interface Overview", available at: https://sqlite.org/cintro.html
8. Medium, "Practical Guide for SQLite and C", available at: https://medium.com/@duthujoe/sqlite-c-a-beginners-guide-243beb3fb3de
9. StackOverflow, "Memory management in SQLite (Transient vs Static)", available at: https://stackoverflow.com/questions/1229102/when-to-use-sqlite-transient-vs-sqlite-static
10. StackOverflow, "Parameter binding for SQL statements", available at: https://stackoverflow.com/questions/31745465/how-to-prepare-sql-statements-and-bind-parameters
11. Conan Blog, "Building interfaces with Dear ImGui", available at: https://blog.conan.io/2019/06/26/An-introduction-to-the-Dear-ImGui-library.html
12. Raylib, "API Reference for Graphics Implementation", available at: https://www.raylib.com/cheatsheet/cheatsheet.html
13. Raylib Examples, "Implementing text input fields", available at: https://www.raylib.com/examples/text/loader.html?name=text_input_box
14. Skyed, "[Dear ImGui C++] Create Simple Notifications for Your UI (PopUp)", available at: https://www.youtube.com/watch?v=RVtvTKeAwXQ
15. Google Fonts, "Typography implementation (Tinos)", available at: https://fonts.google.com/specimen/Tinos?query=tino
16. GitHub. Dear ImGui Discussions, "Implementation of multiple fonts and titles", https://github.com/ocornut/imgui/discussions/5771
17. Figma, "Logic and Flowchart design", available at: https://www.figma.com/resource-library/types-of-flow-charts/